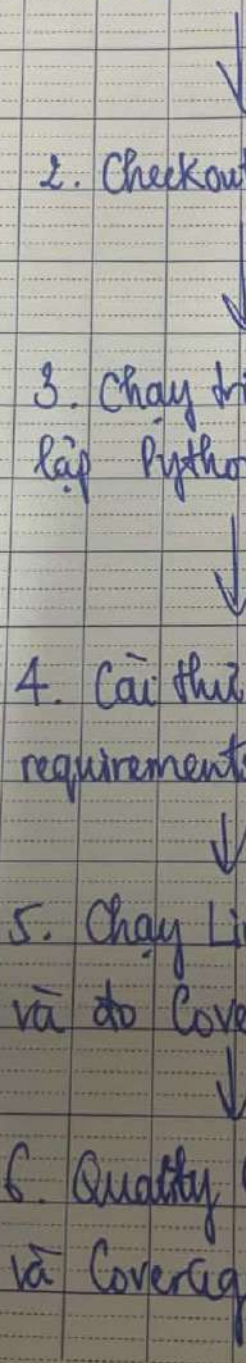
Họ và tên: Tạ Nhật Nguyên – MSSV: 23127104 – Lớp: CTK47B

Ngày thực hành: 16/10/2026 Chữ ký: nguyen

Sơ đồ CI Pipeline cho YummyApp



TK47B

Họ và tên: Tạ Nhật Nguyên – MSSV: 2312704 – Lớp: CTK47B

Ngày thực hành: 16/10/2026 Chữ ký: nguyen

1. Kích hoạt: Github chạy workflow khi có push hoặc pull_request vào nhánh main.
2. Checkout: Tải mã nguồn của repository vào máy chạy CI.
3. Thiết lập môi trường: Dùng runner Ubuntu và cài Python 3.11.
4. Cài thư viện: Cài các dependencies đã ghim phiên bản trong requirements.txt.
5. Kiểm tra: Chạy Linter trước, sau đó chạy Unit Test và đo Coverage.
6. Quality Gate: Nếu test thành công và Coverage đạt ít nhất 80% workflow Pass. Nếu không, workflow Failed.

Tên test case	Hàm được gọi	Đối tượng và giả lập Mocking	Kết quả
Mã GIAM10 hợp lệ khi có mạng	apply_promo_code	Ứng dụng online, tổng tiền 100.000, mã GIAM10	Trả về 90.000
Mất kết nối mạng	apply_promo_code	network_status = "offline"	Phát sinh ConnectionError.
Mã NEW_OS_ONLY	apply_promo_code	os_version = 15; tổng tiền 100.000	Trả về 80.000.
NEW_OS_ONLY trên 14	apply_promo_code	os_version = 14	Phát sinh ValueError.
GPS trong khu vực có nhà	get_nearby_restaurants	Mock GPS trả tọa độ (10.0, 105.0)	Trả về 2 nhà hàng gần
GPS ngoài khu vực có nhà	get_nearby_restaurants	Mock GPS trả tọa độ (20.0, 110.0)	Trả về danh sách rỗng
GPS mất tín hiệu	get_nearby_restaurants	Mock GPS phát sinh GPSSignalLostError	Trả về thông báo Ko xác định vị trí.

FUTUREBOOK

Họ và tên: Đặ Nhật Nguyên – MSSV: 2312704 – Lớp: CTKA7B

Ngày thực hành: 16/10/2026 Chữ ký: nguyen

Câu 2.1: Linter nên chạy trước để phát hiện sớm lỗi cú pháp và một số lỗi tĩnh trong mã nguồn. Nếu phát hiện lỗi, CI có thể dừng ngay lập tức thay vì tiếp tục chạy các bước kiểm thử. Cách làm này giúp tiết kiệm thời gian và thực hiện nguyên tắc fail-fast.

Câu 2.2: Coverage tính tỷ lệ mã nguồn đã được thực thi. Nếu tỷ lệ thấp hơn 80%, lệnh coverage report --fail-under=80 trả về mã thoát khác 0. Trên Linux, mã thoát khác 0 biểu thị kết thúc thất bại. Github Actions nhận trạng thái đó và đánh dấu bước chạy là Failed, khiến workflow bị chặn. Nếu đạt ngưỡng, lệnh trả mã thoát 0 và workflow tiếp tục.

Câu 2.3: Tham số -m hiển thị số dòng chưa được chạy trong tệp. Nhờ đó, kỹ sư kiểm thử có thể xác định phần mã còn thiếu test và bổ sung ca kiểm thử phù hợp để cải thiện Coverage.

Họ và tên: Đỗ Nhật Nguyên — MSSV: 2812704 — Lớp: 01K4TB

Ngày thực hành: 16/10/2026 Chữ ký: nguyen

Câu 3.1: Flaky Test là test có lúc pass, lúc fail dù mã nguồn và điều kiện chạy gần như không đổi. Nguyên nhân có thể kể đến thời gian thực thi không ổn định, xử lý đồng thời hoặc thứ tự chạy test. Lỗi do sai lệch môi trường xảy ra khi test phụ thuộc vào điều kiện bên ngoài như mạng, phiên bản thư viện hoặc thiết bị GPS. Ví dụ, test gọi GPS thật có thể thất bại trên thiết bị không nhận được tín hiệu. Sử dụng mock GPS giúp kiểm thử không phụ thuộc vào phần cứng thật.

Câu 3.2: Ghim phiên bản giúp máy lập trình viên và máy chủ CI cài cùng phiên bản thư viện. Nếu không ghim, lần chạy sau có thể tải 1 phiên bản mới có thể thay đổi về API hoặc cách hoạt động. Khi đó, kết quả kiểm thử có thể khác và lỗi khó tái hiện. Vì vậy, việc nâng cấp thư viện cần được thực hiện có chủ đích và kiểm tra.

Câu 3.3: 1. Tạo một nhánh mới từ nhánh main.

2. Đọc quy chuẩn phát hành và kiểm tra các thay đổi của thư viện

3. Cập nhật phiên bản thư viện trong requirements.txt

4. Chạy Linter, Unit Test và kiểm tra chất lượng.

5. Tạo Pull Request, mô tả thay đổi, và yêu cầu thành viên review.

6. Sửa theo góp ý, chờ CI Pass rồi mới merge; theo dõi sau khi merge và hoàn tác nếu phát sinh lỗi.